Practical no 5: File Handling and Exception Handling in Python

Part A: File Handling

Q1. Write a Python program to create a text file and write data into it.

```
file = open("student.txt", "w")

file.write("Name: Rahul\n")
file.write("Course: B.Sc. CA\n")
file.write("City: Mumbai\n")
file.close()
print("Data written successfully.")
```

Output:

Data written successfully.

The w mode creates a new file. If the file already exists, its previous content is overwritten.

Q2. Write a Python program to read and display the contents of a text file.

```
file = open("student.txt", "r")

data = file.read()

print("File Contents:")
print(data)

file.close()
```

Output:

File Contents:

Name: Rahul
Course: B.Sc. IT
City: Mumbai

Q3. Write a Python program to append new data to an existing text file.

```
file = open("student.txt", "a")

file.write("Marks: 85\n")

file.close()

print("Data appended successfully.")
```

Here, the a mode adds new content without deleting the existing content.

Q4. Write a Python program to read a text file line by line.

```
file = open("student.txt", "r")

for line in file:
    print(line, end="")

file.close()
```

Q5. Write a Python program to demonstrate read(), readline(), and readlines() methods.

```
file = open("student.txt", "r")

print("Using read():")
print(file.read())

file.close()

file = open("student.txt", "r")

print("\nUsing readline():")
print(file.readline())
```

```
file.close()

file = open("student.txt", "r")

print("\nUsing readlines():")
print(file.readlines())

file.close()
```

This program demonstrates three common ways of reading file content.

Q6. Write a Python program to copy the contents of one text file into another text file.

```
source = open("student.txt", "r")
destination = open("copy.txt", "w")

data = source.read()

destination.write(data)

source.close()
destination.close()

print("File copied successfully.")
```

Q7. Write a Python program to count the number of lines, words, and characters in a text file.

```
file = open("student.txt", "r")

data = file.read()

lines = data.split("\n")
words = data.split()
characters = len(data)

print("Number of Lines:", len(lines))
print("Number of Words:", len(words))
```

```
print("Number of Characters:", characters)

file.close()
```

Q8. Write a Python program to explore various attributes of a file object.

```
file = open("student.txt", "r")

print("File Name:", file.name)
print("File Mode:", file.mode)
print("Is File Closed?", file.closed)

file.close()

print("Is File Closed?", file.closed)
```

Sample Output:

```
File Name: student.txt
File Mode: r
Is File Closed? False
Is File Closed? True
```

This demonstrates common file object attributes such as name, mode, and closed.

Q9. Write a Python program to demonstrate the use of tell() and seek() methods.

```
file = open("student.txt", "r")

print("Initial Position:", file.tell())

data = file.read(5)
print("Data Read:", data)

print("Current Position:", file.tell())

file.seek(0)

print("Position after seek:", file.tell())
```

```
file.close()
```

tell() returns the current file position, while seek() changes the file position.

Part B: Directory Operations

Q10. Write a Python program to display the current working directory.

```
import os

current_directory = os.getcwd()

print("Current Working Directory:")
print(current_directory)
```

Q11. Write a Python program to create a new directory.

```
import os

os.mkdir("MyFolder")

print("Directory created successfully.")
```

Q12. Write a Python program to display all files and folders present in the current directory.

```
import os

items = os.listdir()

print("Files and Directories:")

for item in items:
    print(item)
```

Q13. Write a Python program to check whether a specified file exists.

```
import os

filename = input("Enter File Name: ")

if os.path.exists(filename):
    print("File exists.")
else:
    print("File does not exist.")
```

Part C: Built-in Exception Handling

Q14. Write a Python program to handle division by zero using exception handling.

```
try:
    a = int(input("Enter First Number: "))
    b = int(input("Enter Second Number: "))

    result = a / b

    print("Result:", result)

except ZeroDivisionError:
    print("Error: Cannot divide by zero.")
```

Q15. Write a Python program to handle invalid user input using ValueError.

```
try:
    age = int(input("Enter Your Age: "))

    print("Your Age is:", age)

except ValueError:
    print("Error: Please enter a valid integer.")
```

Q16. Write a Python program to handle multiple built-in exceptions.

```
try:
    a = int(input("Enter First Number: "))
    b = int(input("Enter Second Number: "))

    result = a / b

    print("Result:", result)

except ZeroDivisionError:
    print("Error: Division by zero is not allowed.")

except ValueError:
    print("Error: Please enter numbers only.")
```

Q17. Write a Python program to demonstrate try, except, else, and finally blocks.

```
try:
    a = int(input("Enter First Number: "))
    b = int(input("Enter Second Number: "))

    result = a / b

except ZeroDivisionError:
    print("Cannot divide by zero.")

except ValueError:
    print("Invalid input.")

else:
    print("Result:", result)

finally:
    print("Program execution completed.")
```

Concept:

- try → Contains code that may generate an exception.
- except → Handles the exception.
- else → Executes when no exception occurs.
- finally → Executes whether an exception occurs or not.

Part D: File Handling with Exceptions

Q18. Write a Python program to handle FileNotFoundError while opening a file.

```
try:
    file = open("data.txt", "r")

    print(file.read())

    file.close()

except FileNotFoundError:
    print("Error: File does not exist.")
```

Q19. Write a Python program to safely read a text file using exception handling.

```
try:
    filename = input("Enter File Name: ")

    file = open(filename, "r")

    data = file.read()

    print("File Contents:")
    print(data)

except FileNotFoundError:
    print("Error: File not found.")

except PermissionError:
    print("Error: Permission denied.")

finally:
    print("File operation completed.")
```

Part E: User-Defined Exceptions

Q20. Write a Python program to create a user-defined exception for validating age.

```
class InvalidAgeError(Exception):
    pass


try:
    age = int(input("Enter Your Age: "))

    if age < 18:
        raise InvalidAgeError

    print("You are eligible.")

except InvalidAgeError:
    print("Error: Age must be 18 or above.")
```

Q21. Write a Python program to demonstrate a user-defined exception with arguments.

```
class InvalidAgeError(Exception):

    def __init__(self, age):
        self.age = age

    def __str__(self):
        return "Invalid Age: " + str(self.age)


try:
    age = int(input("Enter Your Age: "))

    if age < 18:
        raise InvalidAgeError(age)

    print("Valid Age:", age)

except InvalidAgeError as e:
    print(e)
```

Sample Output:

Enter Your Age: 15
Invalid Age: 15

Here, the value age is passed as an **argument to the user-defined exception**.

Q22. Write a Python program to create a user-defined exception for insufficient bank balance.

```
class InsufficientBalanceError(Exception):

    def __init__(self, balance, amount):
        self.balance = balance
        self.amount = amount

    def __str__(self):
        return "Insufficient Balance. Available Balance: " + str(self.balance)


try:
    balance = 5000

    amount = int(input("Enter Withdrawal Amount: "))

    if amount > balance:
        raise InsufficientBalanceError(balance, amount)

    balance = balance - amount

    print("Withdrawal Successful")
    print("Remaining Balance:", balance)

except InsufficientBalanceError as e:
    print(e)
```